automatisation

chap 1 & 2 :

Critères d'évaluation :

1. **Code source** : Open source ou propriétaire.
2. **Architecture** : Avec ou sans agent.
3. **Fonctionnalités** : Orchestration, gestion de configuration, infrastructure as code.
4. **Support cloud** : Compatibilité avec AWS, GCP, Azure, etc.
5. **Type de langage** : Déclaratif ou procédural.

Outils étudiés :

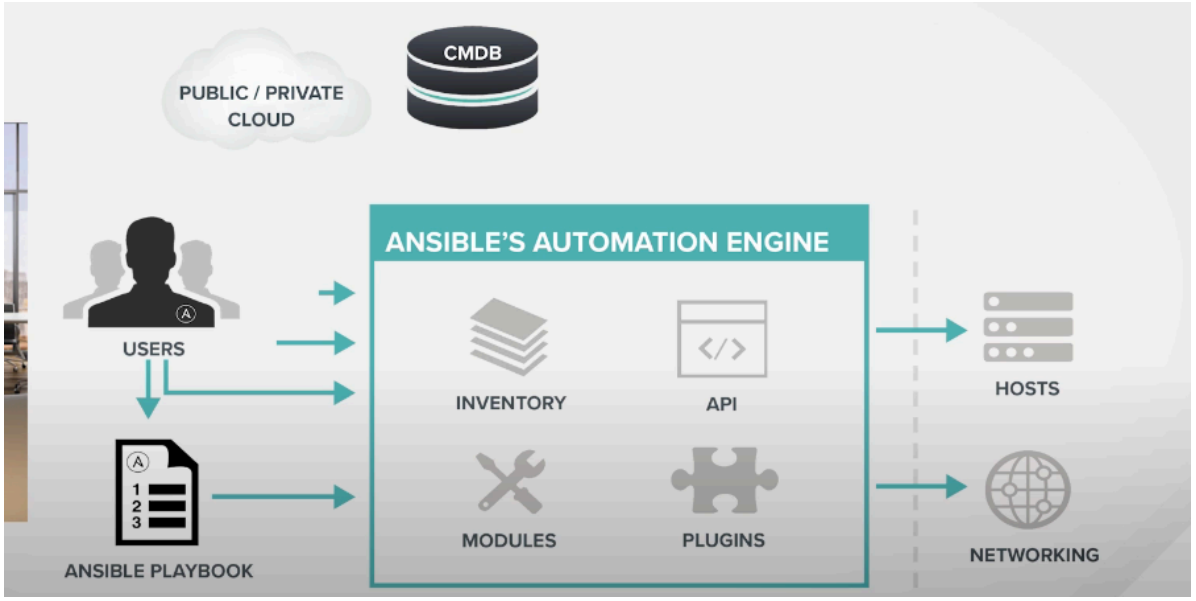
- **Puppet** : Gestion de configuration, architecture Master-Agent, développé en Ruby.
- **Ansible** : Automatisation sans agent, développé en Python, fonctionne en YAML/JSON.
- **Terraform** : Infrastructure as Code, utilise HCL (HashiCorp Configuration Language).

Comparaison rapide :

Outil	Agent ?	Langage	Orchestration	Interface graphique	Cloud supporté
Puppet	Oui	DSL	Non	Non (sauf Enterprise)	AWS, GCP
Ansible	Non	YAML	Oui	Oui (Ansible Tower)	AWS, Azure
Terraform	Non	HCL	Oui	Non	Multi-cloud

---

how ansible works ??



1. **User** writes automation instructions in a **Playbook** (YAML format).
2. The **Playbook** defines tasks using **Modules** (pre-built commands).
3. **Inventory.ini** lists the target **Hosts** (machines to configure).
4. **Plugins** extend Ansible's functionality (e.g., logging, connections).
5. **API** allows integration with other tools.
6. Ansible connects to the **Hosts** (usually via SSH) and executes tasks.
7. The **Hosts** apply configurations as defined in the **Playbook**.

Chap 3 :

- Ansible est un outil open-source d'automatisation qui gère la configuration, le déploiement et l'orchestration via SSH et des playbooks YAML, sans agent.
- Ansible Tower, sa version commerciale, offre une interface avancée, la gestion des rôles, une API REST et des outils de planification.

- L'architecture d'Ansible repose sur trois éléments principaux :

Les nœuds
L'inventaire
Les playbooks

- Un **nœud** dans Ansible est une machine impliquée dans l'automatisation (master/worker)
- L'inventaire d'Ansible regroupe les noeud pour definir l'architecture de l'infrastructure cloud
- Un playbook Ansible est un fichier YAML contenant des "plays" et des tâches qui automatisent des actions sur les nœuds via des modules.

- Un module dans Ansible est une unité de code dédiée à l'exécution de tâches spécifiques sur des nœuds distants.
- Ces modules sont utilisables soit via la ligne de commande (CLI), soit dans un playbook.

En CLI, on appelle directement les modules avec la commande `ansible` pour exécuter des tâches ponctuelles.

- Dans un playbook, ils servent à automatiser des séquences d'opérations complexes.
- Les modules existent sous deux formes : locaux (sur le nœud de contrôle) ou distants (sur les nœuds gérés).
- Les modules locaux s'exécutent sur la machine qui lance Ansible, tandis que les modules distants sont transférés et exécutés sur les machines cibles.

- SSH :
  - garantit des communications sécurisées entre les machines.
  - Cela permet d'exécuter des tâches sur les machines distantes sans installer d'agents supplémentaires.
  - simplifie le déploiement
- Les formats d'inventaire peuvent être en INI ou YAML.
- Le fichier d'inventaire par défaut est /etc/ansible/hosts, mais vous pouvez créer vos propres inventaires.
- Les inventaires sont généralement statiques et ne s'ajustent pas automatiquement aux changements.

Le format INI est simple, direct et facile à rédiger.

Exemple d'inventaire INI :

```
mail.example.com
[webservers]
foo.example.com
bar.example.com
[dbservers]
one.example.com
two.example.com
three.example.com
[test]
test1.example.com
test2.example.com
```

Le format YAML est plus riche, hiérarchique et utile pour des inventaires complexes.

Exemple d'inventaire YAML :

```
all:
  hosts:
    mail.example.com:
  children:
    webservers:
      hosts:
        foo.example.com:
        bar.example.com:
    dbservers:
      hosts:
        one.example.com:
        two.example.com:
        three.example.com:
```

- YAML est sensible à l'indentation, un mauvais espace peut causer des erreurs.
- Il faut utiliser des espaces (pas de tabulations)

Exemple d'erreur d'indentation dans YAML :

```
all:
  hosts:
      server1: # Mauvaise indentation
  server2:
```

Exemple corrigé :

```
all:
  hosts:
    server1:
    server2:
```

- Le format INI est simple et rapide, tandis que YAML est plus adapté pour des configurations complexes.

Il existe deux groupes par défaut :

- **all** : contient tous les hôtes
- **ungrouped** : contient les hôtes non affectés à des groupes spécifiques.

Exemple d'inventaire INI avec groupes personnalisés :

```
mail.example.com  # Hôte isolé

[webservers]      # Groupe de serveurs web
foo.example.com
```

```
bar.example.com

[dbservers]      # Groupe de serveurs de bases de données
one.example.com
two.example.com
```

- Les hôtes non inclus dans un groupe sont placés dans **ungrouped**, et tous les hôtes font partie du groupe **all**.

Dans un inventaire YAML, la création de groupes personnalisés se fait en suivant cette structure :

```
all:
  hosts:
    mail.example.com:
  children:
    webservers:
      hosts:
        foo.example.com:
        bar.example.com:
    dbservers:
      hosts:
        one.example.com:
        two.example.com:
        three.example.com:
    test:
      hosts:
        test1.example.com:
        test2.example.com:
```

- Chaque ligne se termine par deux-points (`:`).
- Le groupe principal `all` englobe toutes les machines définies dans le fichier.
- Le mot-clé `hosts:` liste les machines isolées, non associées à un autre groupe spécifique.
- Pour organiser les machines en groupes personnalisés (comme `webservers`, `dbservers`, ou `test`), on utilise `children:` pour définir des sous-groupes sous `all`.
- Chaque sous-groupe utilise `hosts:` pour lister les machines de ce groupe spécifique.

Imbrication des groupes dans un fichier INI

Dans un fichier INI, l'imbrication des groupes se fait avec la directive `[nom_du_groupe:children]`. Cela permet de définir un groupe contenant d'autres sous-groupes comme enfants.

Exemple :

```
[webservers:children]
foo.example.com
bar.example.com
```

```
[rayen]
server1 ansible_host=192.168.1.10
server2 ansible_host=192.168.1.11

[mokhtar]
server3 ansible_host=192.168.1.12
server4 ansible_host=192.168.1.13

[rayen:children]
mokhtar
```

Imbrication des groupes dans un fichier YAML

Dans un fichier YAML, l'imbrication se fait en utilisant le mot-clé `children:` pour définir des sous-groupes à l'intérieur d'un groupe parent.

Exemple :

```
all:
  children:
    webservers:
      hosts:
        foo.example.com:
        bar.example.com:
```

Un Playbook dans Ansible

Un Playbook peut contenir plusieurs **Plays**. Un **Play** est une section qui définit les éléments suivants :

- **Nom du Play**
- `become:` : Détermine si les tâches doivent s'exécuter avec des privilèges administratifs (`true` ou `false`).
- **Les machines cibles (hosts / groupe)**
- **Les tâches**

---

3. Exemples de Modules Ansible :

- **Module apt** (pour gérer les paquets sur Debian) :
  Exemple : Installer Apache.

  ```
  - name: Installer Apache
    apt:
      name: apache2
      state: present
  ```

- **Module service** (pour gérer les services) :
  Exemple : Démarrer Apache et l'activer au démarrage.

  ```
  - name: Démarrer Apache
    service:
  ```

```
    name: apache2
    state: started
    enabled: yes
```

- **Module copy** (pour copier des fichiers) :
  Exemple : Copier un fichier de configuration.

```
- name: Copier le fichier de configuration
  copy:
    src: /etc/ansible/app_config.conf
    dest: /etc/app_config.conf
```

- **Module file** (pour gérer les fichiers) :
  Exemple : Créer un fichier `test.txt`.

```
- name: Créer un fichier
  file:
    path: /tmp/test.txt
    state: touch
```

4. Définir des Variables dans les Playbooks Ansible :

Les variables sont définies dans la section `vars` d'un play. Elles sont limitées à ce play spécifique. Pour partager des variables entre plusieurs plays, utilisez `vars_files` pour inclure un fichier externe contenant les variables.

Exemple :

```
vars_files:
  - /path/to/variables.yml
```

5. Handlers en Ansible :

Les **handlers** sont des tâches déclenchées en réponse à des changements effectués lors de l'exécution d'un playbook

exp : si un fichier est modifié , redemarre un service :

- **Définition d'un handler** :

```
handlers:
  - name: Restart Apache
    service:
      name: apache2
      state: restarted
```

- **Notifier un handler** dans une tâche :

```
tasks:
  - name: Modifier le fichier de configuration
    copy:
      src: /etc/ansible/config.conf
      dest: /etc/apache2/config.conf
    notify: Restart Apache
```

Ainsi, un handler ne sera exécuté que si une tâche modifie l'état de l'hôte, et il ne sera exécuté qu'une seule fois par playbook, évitant ainsi la répétition inutile d'opérations comme le redémarrage d'un service.

Principes des Handlers Ansible

1. **Déclenchement Sélectif** : Les handlers réagissent à des événements spécifiques (ex : modification de fichier).
2. **Référencé par Plusieurs Tâches** : Un handler peut être appelé par plusieurs tâches.
3. **Exécution Unique** : Chaque handler est exécuté une seule fois, même s'il est notifié plusieurs fois.
4. **Ordre d'Exécution** : Les handlers s'exécutent **après toutes les tâches principales**, dans l'ordre où ils sont définis.

Exemple

Si deux tâches créent des fichiers, chacune notifiant le même handler, ce dernier ne sera exécuté qu'une seule fois après la fin des tâches, affichant un message.

Module template vs copy

- **template** : Copie un fichier **Jinja2** et remplace les variables (ex : `{{ ansible_user }}`) par leurs valeurs, avec des permissions définies.
- **copy** : Copie un fichier statique sans modification.

Exemple Template

```
- name: Copier un fichier de configuration
  template:
    src: /path/to/file.j2
    dest: /home/worker/config_file
    owner: worker
    mode: '0640'
```

Le fichier est copié avec les variables remplacées et les bonnes permissions.

Conclusion

- **template** permet une personnalisation dynamique.
- **copy** est simple et ne permet pas de transformations.

---

les boucles ?

- répéter une tâche plusieurs fois avec différentes valeurs
- réduisant le risque d'erreurs humaines
- améliorant la lisibilité du code.

Loop :

```yaml
---
- name: Playbook pour créer 3 nouveaux utilisateurs
  hosts: worker
  become: true
  tasks:
    - name: Créer plusieurs utilisateurs
      user:
        name: "{{ item }}"
        state: present
      loop:
        - user1
        - user2
        - user3
```

Condition :

```yaml
---
- name: Installer le serveur web Apache sur les nœuds workers
  hosts: worker
  become: yes
  tasks:
    - name: Installer Apache sur Ubuntu
      apt:
        name: apache2
        state: present
      when: ansible_os_family == "Debian"

    - name: Installer Apache sur CentOS
      yum:
        name: httpd
        state: present
      when: ansible_os_family == "RedHat"

    - name: Afficher un simple message
      debug:
        msg: "Le playbook a continué à s'exécuter"
```

- Vérifier la syntaxe :

  `ansible-playbook -i mon_inventaire.ini mon_playbook.yml --syntax-check`

- Simulation sans appliquer les changements :

  `ansible-playbook -i mon_inventaire.ini mon_playbook.yml --check`

- Lister les tâches sans les exécuter :

  `ansible-playbook -i mon_inventaire.ini mon_playbook.yml --list-tasks`

- Exécuter uniquement les tâches avec un tag :

  `ansible-playbook -i mon_inventaire.ini mon_playbook.yml --tags ftp`

Commandes Ad-Hoc Ansible

Les commandes ad-hoc permettent d'exécuter des tâches rapides sans écrire un playbook.

Exemple d'Utilisation des Commandes Ad-Hoc

- **Ping** : Vérifier la connectivité SSH avec une machine.
- **Setup** : Collecter des informations système.
- **User** : Créer un utilisateur.
- **Install Package** : Installer un paquet sur un hôte.

L'idempotence dans Ansible

L'idempotence garantit qu'un playbook peut être exécuté plusieurs fois sans modifier l'état d'un système si celui-ci est déjà conforme à la configuration souhaitée, assurant ainsi une stabilité du système. Si tout est déjà en place, Ansible ne fait rien !

Résumé suite chap 3:

Dans l'automatisation et la gestion des infrastructures, la sécurité des données sensibles est cruciale. **Ansible Vault** permet de chiffrer des informations confidentielles (mots de passe, clés API, certificats) pour éviter leur divulgation.

Fonctionnement d'Ansible Vault

- Utilise un chiffrement symétrique (même clé pour chiffrer/déchiffrer).
- Peut protéger :
  - Variables sensibles (mots de passe, clés secrètes).
  - Fichiers entiers (playbooks, inventaires).
  - Rôles et templates Ansible contenant des données sensibles.

Principales commandes :

- **Chiffrer un fichier** : `ansible-vault encrypt fichier.yml`
- **Déchiffrer un fichier** : `ansible-vault decrypt fichier.yml`
- **Afficher un fichier chiffré** : `ansible-vault view fichier.yml`
- **Modifier un fichier chiffré** : `ansible-vault edit fichier.yml`
- **Créer un fichier déjà chiffré** : `ansible-vault create fichier.yml`
- **Changer le mot de passe Vault** : `ansible-vault rekey fichier.yml`
- **Utiliser plusieurs mots de passe Vault** : `ansible-vault encrypt fichier.yml --vault-id dev@prompt`

Exécution d'un playbook chiffré

Un playbook chiffré doit être exécuté avec la clé de déchiffrement via `--ask-vault-pass`, sinon Ansible affichera une erreur.

Commande :

`ansible-playbook -i myinventory playbook.yml --ask-vault-pass`

Chiffrement d'une variable dans un playbook

1. Déplacer la variable dans un fichier `.yml` (ex: `secrets.yml`).
2. Chiffrer le fichier avec `ansible-vault encrypt secrets.yml`.
3. Référencer le fichier dans le playbook via `vars_files`.
4. Exécuter avec `-ask-vault-pass` pour déchiffrer à l'exécution.

Fournir le mot de passe Vault

- **`-ask-vault-pass`** : Demande le mot de passe à chaque exécution.
- **`-vault-password-file`** : Charge le mot de passe depuis un fichier.
- **`-vault-id`** : Permet d'utiliser plusieurs mots de passe pour différents fichiers.

Chap 4 : gestion des variables

Règles de dénomination des variables :

1. **Caractères autorisés** : lettres, chiffres et traits de soulignement (_).
2. **Commencement** : un nom de variable doit commencer par une lettre ou un trait de soulignement.
3. **Réservations** : certaines chaînes comme les mots-clés Python ou Playbook (par exemple, `async`, `environment`) ne sont pas valides.
4. **Bonnes pratiques** : les noms doivent être significatifs et courts pour faciliter le débogage.

Exemples valides : `web_server_port`, `db_connection_timeout`, `max_attempts_retry`.

Exemples invalides : `tm,@p%$2` (caractères spéciaux), `1var` (commence par un chiffre).

Référencement de variables complexes :

1. **Listes** :
   - Définir une liste :

     ```
     vars:
       region:
         - northeast
         - southeast
         - midwest
     ```

   - Référencer la liste entière : `{{ regions }}`
   - Référencer un élément spécifique : `{{ regions[1] }}` (affiche "southeast").
2. **Dictionnaires** :
   - Définir un dictionnaire :

     ```
     vars:
       users:
         admin: carole
         developer: peter
         tester: sophie
     ```

   - Référencer tout le dictionnaire : `{{ users }}`
   - Référencer une clé spécifique : `{{ users.admin }}`
3. **Structures imbriquées** (Listes de dictionnaires) :
   - Définir une structure imbriquée :

     ```
     vars:
       users:
     ```

```
  - name: carole
    role: admin
    department: IT
  - name: peter
    role: tester
    department: QA
  - name: sophie
    role: developer
    department: Dev
```

- Référencer toute la structure imbriquée : `{{ users }}`
- Référencer une valeur spécifique : `{{ users[2].role }}` (affiche "developer").

Les structures imbriquées permettent de combiner des listes et des dictionnaires pour gérer des données complexes.

Types de variables dans Ansible

Extra Vars :

Une des nombreuses possibilités pour définir des variables consiste à les fournir en argument en ligne de commande avec l'option -e (ou --extra-vars).

exp:

ansible-playbook -i my_inventory.ini myplaybook.yml --extra-vars "ma_variable=Arctic7"

Task Vars :

Task Vars sont des variables locaux

Exemple pratique :

```
- name: Exemple de playbook avec Task Vars
  hosts: localhost
  tasks:
    - name: Définir une variable locale pour cette tâche
      vars:
        port: 8080
      debug:
        msg: "Le port utilisé est {{ port }}"

    - name: Une autre tâche sans l'impact de la première
      debug:
        msg: "Pas de variable locale ici"
```

Variable partagée :

Example:

```
- name: Exemple de playbook avec variable partagée
  hosts: localhost
  vars:
    shared_variable: "This is a shared variable"

  tasks:
    - name: Afficher la variable partagée dans la première tâche
      debug:
        msg: "La variable partagée est : {{ shared_variable }}"

    - name: Utiliser la variable partagée dans une deuxième tâche
      debug:
        msg: "Encore une fois, la variable partagée : {{ shared_variable }}"
```

Variables Enregistrées (Registered Vars) :

Les **variables enregistrées** permettent de stocker la sortie d'une tâche afin de pouvoir la réutiliser dans d'autres tâches du même playbook. Cela permet de récupérer des informations générées par une tâche et de les utiliser dans le reste du playbook.

Exemple pratique :

```
- name: Exemple de playbook avec une variable enregistrée
  hosts: localhost
  tasks:
    - name: Exécuter une commande et enregistrer sa sortie
      command: echo "Hello, Ansible!"
      register: result

    - name: Utiliser la sortie de la commande
      debug:
        msg: "La commande a renvoyé : {{ result.stdout }}"
```

Variables Facts :

Les **variables facts** sont des informations dynamiques collectées par Ansible depuis les machines cibles (workers).

Collecte des Facts :

- **Automatique** : Par défaut
- **Désactivation** : Vous pouvez désactiver cette collecte automatique en définissant `gather_facts: no` dans le playbook.

Exemple pratique :

```
- name: Exemple d'utilisation des variables facts
  hosts: localhost
  gather_facts: yes
  tasks:
    - name: Afficher la version du noyau
      debug:
        msg: "La version du noyau est : {{ ansible_facts.kernel }}"

    - name: Afficher l'espace disque disponible
      debug:
        msg: "Espace libre sur le disque : {{ ansible_facts.disk_free }}"
```

Variable fact	Description	Exemple d'utilisation
ansible_facts	Contient toutes les facts collectées par Ansible sous forme de dictionnaire.	{{ ansible_facts }}
ansible_os_family	Famille du système d'exploitation (par exemple, Debian, RedHat).	{{ ansible_os_family }}
ansible_distribution	Nom de la distribution du système d'exploitation (par exemple, Ubuntu, CentOS).	{{ ansible_distribution }}
ansible_distribution_version	Version de la distribution du système d'exploitation.	{{ ansible_distribution_version }}
ansible_disk	Liste des disques et partitions disponibles sur la machine cible.	{{ ansible_disks }}
ansible_memfree_mb	Mémoire libre disponible sur la machine cible en mégaoctets.	{{ ansible_memfree_mb }}
ansible_default_ipv4.address	Adresse IP par défaut de la machine cible (en IPv4).	{{ ansible_default_ipv4.address }}
ansible_default_ipv6.address	Adresse IP par défaut de la machine cible (en IPv6).	{{ ansible_default_ipv6.address }}
ansible_default_ipv4.macaddress	Adresse MAC de la machine cible (en IPv4).	{{ ansible_default_ipv4.macaddress }}
ansible_default_ipv4.gateway	Passerelle par défaut de la machine cible (en IPv4).	{{ ansible_default_ipv4.gateway }}
ansible_interfaces	Liste des interfaces réseau disponibles.	{{ ansible_interfaces }}

Variables Magiques :

Les **variables magiques** sont des variables prédéfinies générées automatiquement par Ansible pendant l'exécution d'un playbook. Elles ne dépendent pas de la collecte des facts via `gather_facts` et sont indépendantes des informations détaillées sur le système.

Indépendance par rapport à `gather_facts` :

Même si `gather_facts: no` est activé et qu'Ansible ne collecte pas de facts détaillés, les variables magiques continueront de fonctionner. Elles sont basées sur l'inventaire, les groupes et l'état d'exécution du playbook.

Exemples d'utilisation des Variables Magiques :

- `inventory_hostname` : Nom ou adresse IP de l'hôte en cours de traitement.
- `group_names` : Liste des groupes auxquels l'hôte appartient.

Exemple pratique :

```
- name: Exemple de playbook avec variables magiques
  hosts: localhost
  gather_facts: no
  tasks:
    - name: Afficher le nom de l'hôte
      debug:
        msg: "L'hôte traité est : {{ inventory_hostname }}"

    - name: Afficher les groupes de l'hôte
      debug:
        msg: "Les groupes auxquels cet hôte appartient sont : {{ group_names }}"
```

Inventory Vars :

Les **variables d'inventaire** permettent d'attribuer des variables spécifiques à un hôte ou à un groupe d'hôtes dans l'inventaire d'Ansible. Il existe plusieurs méthodes pour les définir :

1. **Dans le fichier d'inventaire**
   - Format **INI** : Vous pouvez associer des variables directement à un hôte ou à un groupe d'hôtes.
   - Format **YAML** : Vous pouvez structurer les variables pour un hôte ou un groupe de manière plus lisible.
2. **Dans les répertoires `host_vars` et `group_vars`**
   - Plutôt que de surcharger l'inventaire avec des variables, il est recommandé de les organiser dans des fichiers dédiés dans les répertoires `host_vars` (pour des hôtes spécifiques) et `group_vars` (pour des groupes d'hôtes).

Exemple pratique :

1. **Format INI** :

```
[web_servers]
web1 ansible_ssh_host=192.168.1.1
web2 ansible_ssh_host=192.168.1.2
```

```
[web_servers:vars]
environnement=production
```

Dans cet exemple :

- Les hôtes `web1` et `web2` appartiennent au groupe `web_servers`.
- La variable `environnement=production` est appliquée à tous les hôtes du groupe `web_servers`.

1. **Format YAML** :

```
web_servers:
  hosts:
    web1:
      ansible_ssh_host: 192.168.1.1
    web2:
      ansible_ssh_host: 192.168.1.2
  vars:
    environnement: production
```

1. **Dans les répertoires `host_vars` et `group_vars`** :
   - **Variables pour un hôte** (ex. `host_vars/web1.yml`) :

     ```
     classe: frontend
     ```

   - **Variables pour un groupe** (ex. `group_vars/web_servers.yml`) :

     ```
     environnement: production
     ```

Ces approches permettent une gestion plus organisée et modulable des variables dans les playbooks Ansible.

Résumé : Role default vars vs Role vars dans Ansible

Les rôles dans Ansible permettent d'organiser les tâches en séparant les différentes configurations. Deux fichiers principaux servent à définir des variables :

1. `defaults/main.yml`
   - Contient des **valeurs par défaut** qui peuvent être remplacées par l'utilisateur.
   - Ces variables sont flexibles et destinées à être personnalisées.

   Exemple :

   ```
   # defaults/main.yml
   web_port: 8080
   ```

   👉 L'utilisateur peut redéfinir `web_port` dans l'inventaire ou le playbook.

2. `vars/main.yml`
   - Contient des **valeurs internes au rôle** qui ne doivent **pas être modifiées** par l'utilisateur.
   - Utilisé pour des paramètres critiques nécessaires au bon fonctionnement du rôle.

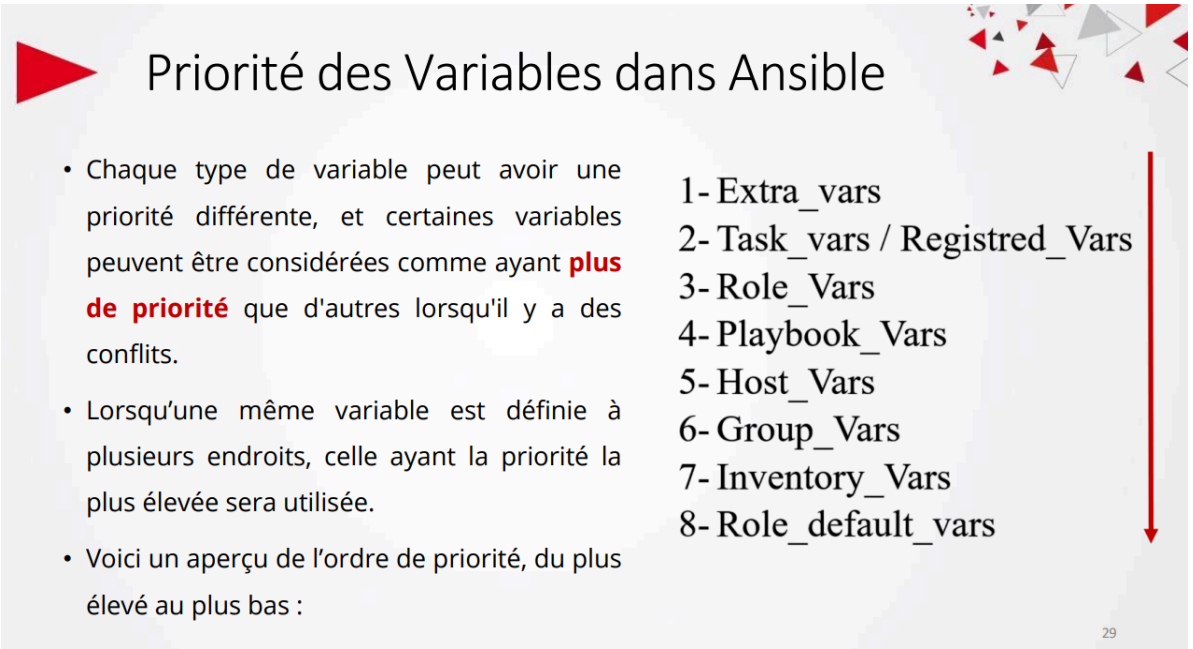
   Exemple :

   ```
   # vars/main.yml
   config_file_path: /etc/myapp/config.yml
   ```

   👉 L'utilisateur **ne doit pas modifier** `config_file_path` pour éviter les erreurs.

Synthèse

Fichier	Rôle	Modifiable par l'utilisateur ?
`defaults/main.yml`	Valeurs par défaut du rôle	✅ Oui
`vars/main.yml`	Variables critiques internes	❌ Non



Chap 5 : terraform

- L'Infrastructure-as-Code (IaC) automatise la gestion des infrastructures via des fichiers de configuration.

Caractéristiques des outils IaC

Approches IaC

Ansible => Mutable = Modifie la configuration en place sur des serveurs existants.
Terraform => Immuable = Détruit et recrée les ressources pour appliquer les changements.

L'IaC adopte une approche **déclarative** ou **impérative**.

- **Déclarative** : définit l'état final souhaité, l'outil se charge de l'atteindre automatiquement.
- **Impérative** : spécifie les commandes à exécuter dans un ordre précis pour obtenir la configuration voulue.
  La plupart des outils privilégient l'approche déclarative pour plus de simplicité.

Mutable vs. Immuable

- **Mutable** : l'infrastructure est modifiée en continu, nécessitant des mises à jour fréquentes.
- **Immuable** : chaque mise à jour crée une nouvelle instance, assurant stabilité et cohérence.

Modèles de communication

- **Agent-Based vs. Agentless** : certains outils nécessitent un agent installé sur les cibles, d'autres non.
- **Push vs. Pull** : soit l'outil envoie les configurations aux cibles (*Push*), soit les cibles récupèrent elles-mêmes leurs paramètres (*Pull*).

Avantages de l'Infrastructure as Code (IaC)

- **Réduction des coûts** : Moins d'interventions humaines, automatisation des tâches et optimisation des ressources cloud.
- **Déploiements accélérés** : Provisionnement rapide des infrastructures en quelques minutes.
- **Moins d'erreurs** : Configuration versionnée et testée pour éviter les pannes.
- **Cohérence des environnements** : Uniformité entre développement, test et production.
- **Élimination des écarts de configuration** : Chaque serveur est identique, facilitant la maintenance.

En bref : l'IaC apporte automatisation, rapidité, fiabilité et standardisation, libérant du temps pour l'innovation.

Terraform : Vue d'ensemble

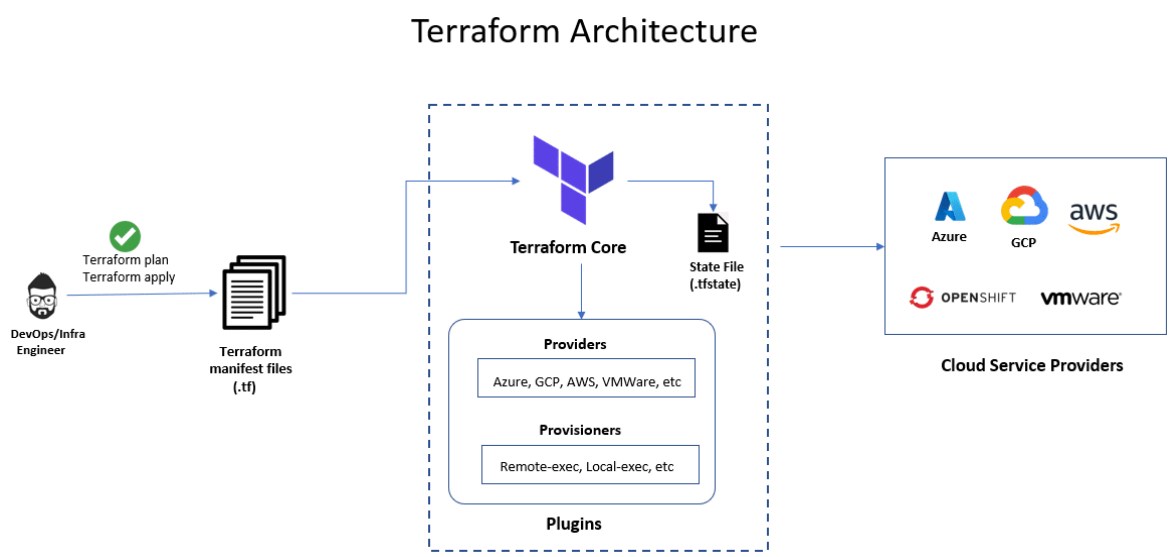
Terraform est un outil open source d'Infrastructure as Code (IaC) qui automatise la gestion des infrastructures. Il suit une approche **déclarative**, où l'utilisateur définit l'état souhaité, et Terraform l'applique.

Il utilise **HCL (HashiCorp Configuration Language)**, un langage simple et lisible. Son principal avantage est sa **compatibilité multi-provider**, prenant en charge plus de 125 fournisseurs comme AWS, Azure et Google Cloud.

Syntaxe HCL :

- Commentaires : `#` (une ligne) ou `/* */` (plusieurs lignes).
- Variables : `clé = valeur` (string, number, boolean, list, map).

- Chaînes : entre `""`.
- Booléens : `true` ou `false`.



Architecture Terraform

Terraform Core

Terraform Core orchestre l'infrastructure en s'appuyant sur :

1. **Fichiers de configuration (.tf)** : définissent les ressources et leurs propriétés.
2. **Terraform State (terraform.tfstate)** : stocke l'état actuel des ressources déployées.

Il compare la configuration souhaitée à l'état actuel et applique les changements nécessaires.

Terraform State

- Stocké localement ou à distance (S3, Terraform Cloud) pour la collaboration.
- Permet de détecter les modifications faites hors de Terraform.

Plugins Terraform

1. **Providers** (essentiels) : connectent Terraform aux services comme AWS, Azure, GCP, Docker.
2. **Provisioners** (optionnels) : exécutent des scripts après déploiement (`remote-exec`, `local-exec`).

⚠️ Les provisioners ne fonctionnent que sur des ressources créées par Terraform. Pour les autres, privilégier Ansible.

Déploiement d'une instance EC2 sur AWS avec Terraform

1️⃣ Déclaration des dépendances

Définition du provider AWS dans `main.tf` :

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 4.0"
    }
  }
}
```

✅ Permet d'assurer la compatibilité avec une version spécifique du provider.

2️⃣ Configuration du Provider AWS

```
provider "aws" {
    region = "us-east-2"
    access_key = "VOTRE_CLÉ"
    secret_key = "VOTRE_CLÉ_SECRÈTE"
}
```

⚠️ **Ne jamais stocker les clés en dur** → Utiliser des variables d'environnement.

3️⃣ Création de la ressource EC2

```
resource "aws_instance" "my_ec2_instance" {
    ami = "ami-07c1207a9d40bc3bd"
    instance_type = "t2.micro"
}
```

✅ `ami` → ID de l'image (Ubuntu 18.04)

✅ `instance_type` → Type d'instance (t2.micro)

4️⃣ Organisation du code

- `main.tf` : Déclare les ressources
- `variables.tf` : Définit les variables
- `output.tf` : Affiche des informations utiles

`variables.tf` :

```
variable "region" {
  default = "us-east-2"
}
variable "instance_type" {
```

```
  default = "t2.micro"
}
```

`output.tf` :

```
output "instance_ip" {
  value = aws_instance.my_ec2_instance.public_ip
}
```

✅ Permet d'afficher l'IP publique de l'instance après déploiement.

🚀 **Prochaine étape** : Appliquer Terraform pour déployer l'instance !

Workflow de Terraform 🚀

Terraform suit **5 étapes clés** pour gérer l'infrastructure de manière efficace :

1️⃣ `terraform init` (Initialisation)

🔷 Prépare l'environnement Terraform.

🔷 Télécharge les **providers** et **plugins nécessaires**.

🔷 Initialise le répertoire de travail.

```
terraform init
```

2️⃣ `terraform validate` (Validation)

🔷 Vérifie la **syntaxe** des fichiers `.tf`.

🔷 S'assure que toutes les ressources sont bien définies.

```
terraform validate
```

✅ Détecte les erreurs **avant** d'exécuter le plan.

3️⃣ `terraform plan` (Création du plan)

🔷 Compare l'état actuel de l'infrastructure avec celui défini dans les fichiers `.tf`.

🔷 Affiche les modifications qui seront **appliquées**.

```
terraform plan
```

✅ **Aucune ressource n'est modifiée** à cette étape.

4️⃣ `terraform apply` (Application des changements)

🔷 Applique le plan et **crée/modifie/supprime** les ressources.

🔷 Met à jour le **Terraform State** pour rester synchronisé.

```
terraform apply
```

⚠️ **Étape critique** : Vérifier avant d'exécuter !

5️⃣ `terraform destroy` (Destruction des ressources)

🔷 Supprime toutes les ressources créées.

🔷 Nettoie l'environnement pour éviter des **coûts inutiles**.

```
terraform destroy
```

✅ **Utile pour les environnements de test** ou la suppression complète de l'infrastructure.

💡 **Astuce** : Toujours exécuter `terraform plan` avant `terraform apply` pour éviter des erreurs imprévues ! 😎

Terraform vs Ansible 🤔 💡

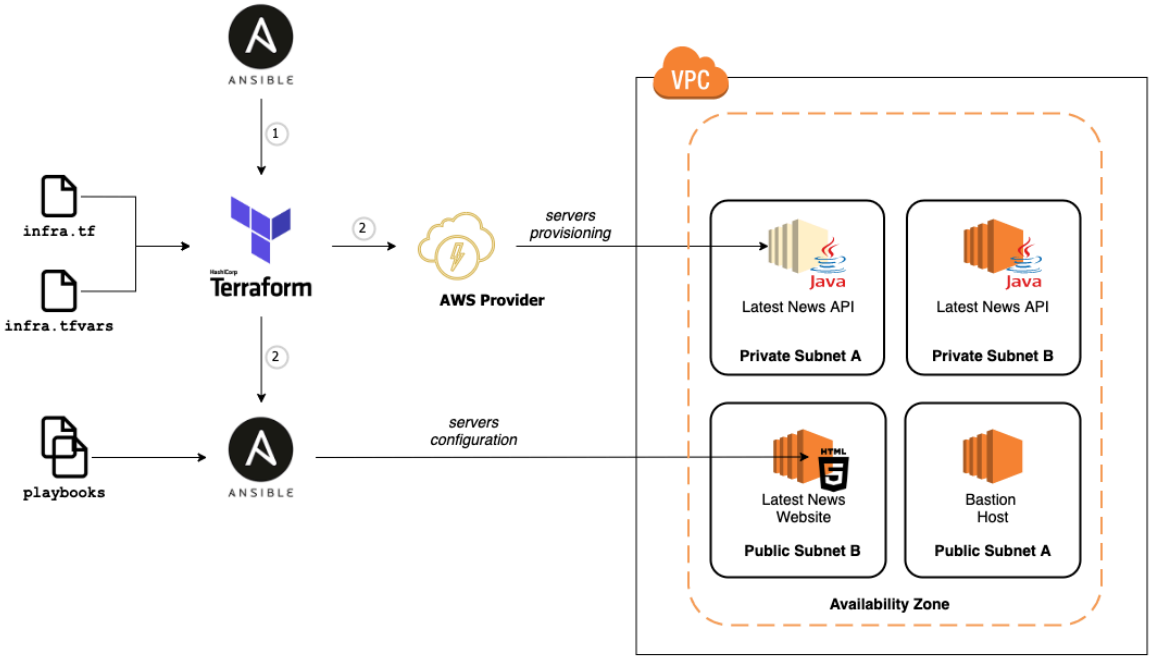
🚀 Terraform	🔧 Ansible
Gère l'infrastructure 🏗️	**Configure et automatise** 🔄
Crée des **serveurs, réseaux, bases de données** 🌐	Installe des logiciels, modifie des fichiers, applique des mises à jour 💻
Déclaratif (définit l'état final) 📜	**Impératif** (exécute des tâches étape par étape) 🛠️
Utilisé pour **provisionner** 🌱	Utilisé pour **configurer et maintenir** ⚙️
Exemple : **Créer une VM sur AWS** ☁️	Exemple : **Installer Apache sur une VM** 🔥

💡 **Ensemble, ils forment un combo puissant** :

➡️ **Terraform** pour **créer** l'infrastructure.

➡️ **Ansible** pour **configurer et gérer** les systèmes.

🔥 **Terraform + Ansible** = Automatisation ultime 🚀



Terraform + Ansible : Collaboration Parfaite 🌐🤝

Terraform 🏗️

- **Création de l'infrastructure**
  - Provisionnement des ressources : serveurs, VPC, sous-réseaux, etc.
  - Déploiement d'instances, réseaux et autres éléments d'infrastructure cloud.

Ansible 🔧

- **Configuration des systèmes**
  - Installation de logiciels, configuration de serveurs, gestion des mises à jour.
  - Déploiement et configuration d'applications sur les serveurs provisionnés par Terraform.

💡 **Processus** :

1. **Terraform** déploie l'infrastructure (serveurs, réseaux).
2. **Ansible** prend le relais pour configurer ces ressources et les rendre prêtes à l'emploi.

Ensemble : Ces outils créent une solution complète pour gérer l'infrastructure et son cycle de vie. 🚀

BONNE CHANCE 🙂